

The roadmap should gradually move them from **syntax** → **logic** → **scenarios** → **decomposition** → **optimization** → **professional SQL thinking**.

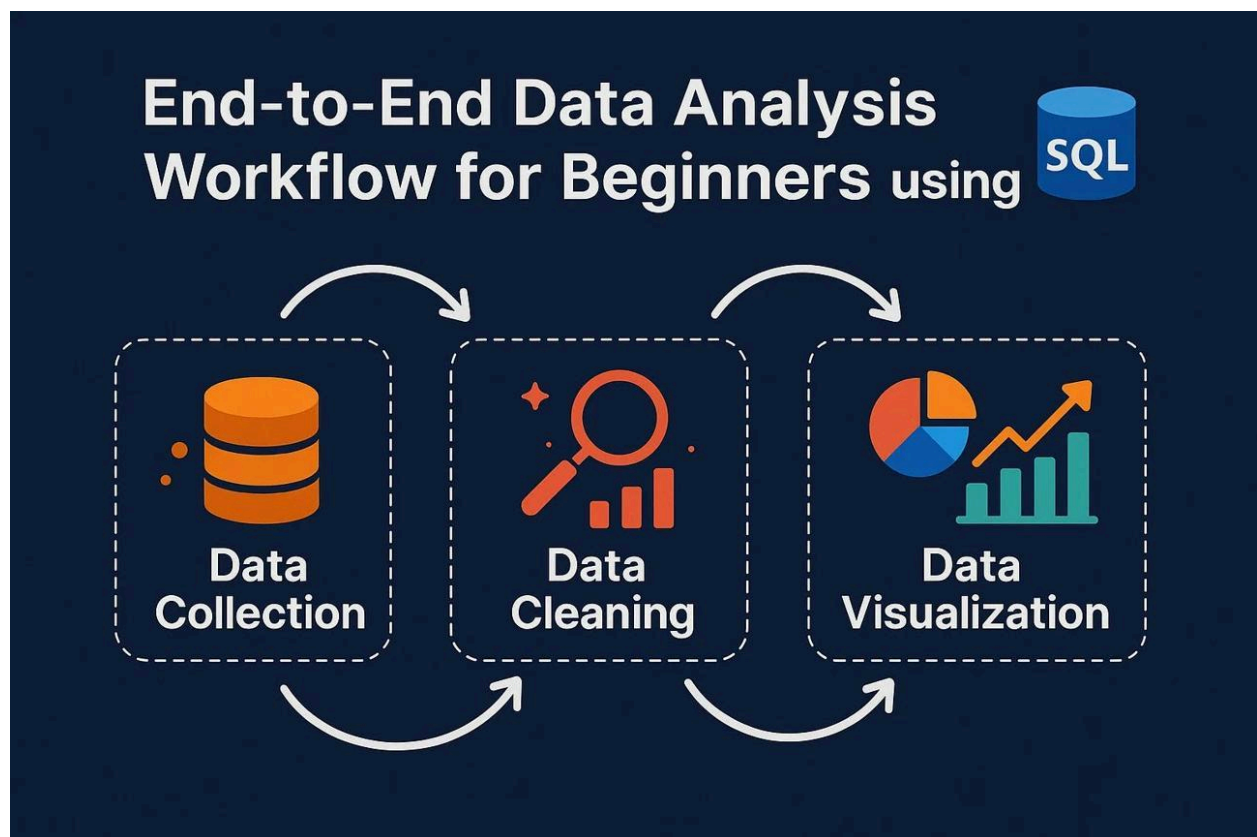
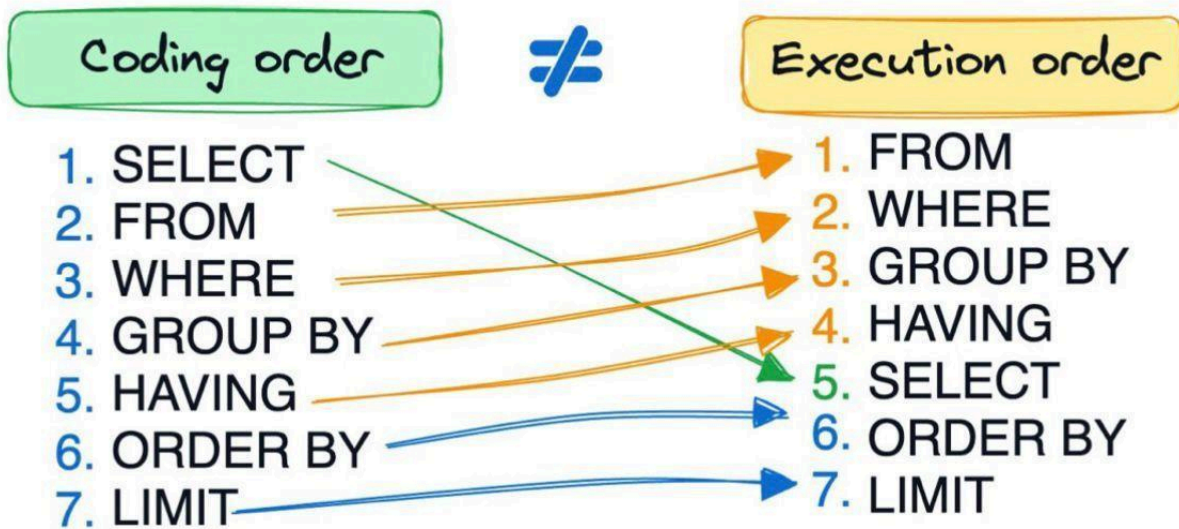
SQL Problem-Solving Roadmap

SQL

Beginner to Advance

Roadmap

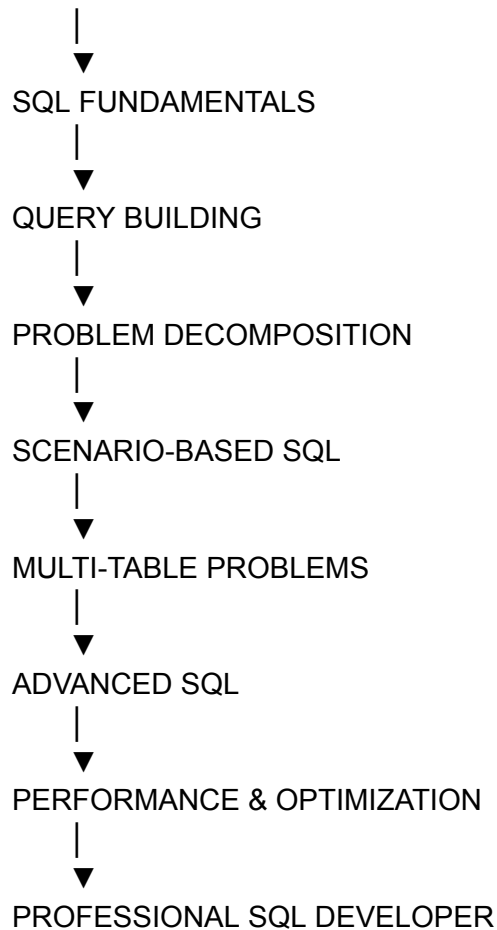
Beginner	Intermediate	Advance
• SELECT statement • WHERE clause • ORDER BY • LIMIT/TOP • INSERT INTO • UPDATE • DELETE • Basic data types • CREATE TABLE • DROP TABLE • ALTER TABLE • Primary keys • Foreign keys • NOT NULL • constraint • AND, OR, NOT operators • IN operator • BETWEEN operator • LIKE operator • IS NULL/IS NOT NULL • DISTINCT • COUNT, SUM, AVG, MIN, MAX • COUNT, SUM, AVG, MIN, MAX	• INNER JOIN • LEFT JOIN • RIGHT JOIN • FULL OUTER JOIN • Self joins • CROSS JOIN • GROUP BY • HAVING clause • Subqueries • UNION, UNION ALL • INTERSECT • EXCEPT/MINUS • CASE statements • Date functions • String functions • Numeric functions • Type casting/conversion • CHECK constraint • CREATE INDEX • DROP INDEX • Views (CREATE VIEW) • Multiple table joins • Aggregate functions with GROUP BY • COALESCE • NULLIF	• Window functions (ROW_NUMBER, RANK, DENSE_RANK) • PARTITION BY • Common Table Expressions (CTEs) • Recursive CTEs • Correlated subqueries • EXISTS and NOT EXISTS • Stored procedures • Triggers • Transactions (BEGIN, COMMIT, ROLLBACK) • Indexes (clustered, non-clustered, composite) • Query optimization • Execution plans • Normalization (1NF, 2NF, 3NF, BCNF) • Query retrines • Pivoting and unpivoting • Normalization (1NF, 2NF, 3NF, BCNF) • Execution plans • Execution plans • Normalization (1NF, 2NF, 3NF, BCNF) • Pivoting and unpivoting



5

The Overall Journey

THEORY KNOWLEDGE



The most important principle:

Don't teach them more SQL until they learn how to think through a SQL problem.

PHASE 0 — Diagnose the Real Problem

Before starting the roadmap, give them a **diagnostic assessment**.

Don't tell them what SQL concept each question tests.

For example:

Question 1

Find all customers who placed an order above ₹10,000.

Question 2

Find customers who have never placed an order.

Question 3

Find the second-highest order amount.

Question 4

Find the customer who has spent the most money.

Question 5

Find the top-selling product in each category.

Question 6

Find customers whose spending is higher than the average customer spending.

Question 7

Find customers who placed orders in January but not February.

Observe **how they think**, not just whether the query is correct.

Look for:

- Do they understand the requirement?
- Can they identify tables?
- Can they identify relationships?
- Do they know what the output should look like?
- Do they know which columns matter?
- Can they break the problem into smaller pieces?
- Do they immediately start typing?
- Do they randomly try JOINS?
- Do they use subqueries without understanding why?
- Do they confuse **WHERE** and **HAVING**?
- Do they understand duplicates?

This tells you where the actual problem is.

PHASE 1 — Rebuild the SQL Foundation

Duration: 1–2 weeks

They already know SQL theoretically, so don't spend weeks teaching definitions.

Instead, use **micro-problems**.

1. SELECT

Start with:

```
SELECT customer_name  
FROM customers;
```

Then gradually introduce:

```
SELECT customer_name, city  
FROM customers;
```

Then:

```
SELECT customer_name,  
       price,  
       quantity,  
       price * quantity AS total  
FROM orders;
```

Then introduce:

- aliases
- expressions
- **DISTINCT**
- calculated columns

Teaching principle

Don't say:

"Today we are learning aliases."

Instead:

"The business wants the column to appear as `total_sales`. How would you do it?"

This forces thinking.

2. Filtering

Move from:

WHERE price > 1000

to:

WHERE price > 1000
AND category = 'Electronics';

Then:

WHERE category IN ('Electronics', 'Furniture')

Then:

WHERE price BETWEEN 1000 AND 5000

Then:

WHERE customer_name LIKE 'A%'

Then:

WHERE email IS NULL

Important exercise progression

Don't teach 10 operators and then give exercises.

Instead:

Problem

↓

Think

↓

Choose operator

↓
Write query
↓
Validate result

PHASE 2 — Aggregation Thinking

This is where trainees usually begin struggling.

Teach:

COUNT()
SUM()
AVG()
MIN()
MAX()

But don't stop there.

Move quickly to:

GROUP BY

For example:

How much revenue did each customer generate?

They should think:

What am I measuring?

↓
Revenue

↓
For whom?

↓
Each customer

↓
Therefore GROUP BY customer

Then:

```
SELECT customer_id,  
       SUM(amount) AS total_spent  
FROM orders
```

GROUP BY customer_id;

PHASE 3 — The Most Important Skill: Understand the Question

This should become a **permanent habit**.

Teach trainees to translate every question into five things:

The 5-question method

Before writing SQL, ask:

1. WHAT?

What information do I need?

Example:

"Find customers who spent more than ₹50,000."

Answer:

Customer
Total spending

2. FROM WHERE?

Which table contains the information?

customers
orders

3. HOW?

How are the tables connected?

customers.customer_id
=
orders.customer_id

4. FILTER?

What conditions exist?

total spending > 50,000

5. OUTPUT?

What should the final result look like?

customer_id | customer_name | total_spent

This simple framework will dramatically improve their problem solving.

PHASE 4 — Teach the SQL Mental Model

One of the biggest mistakes in SQL education is teaching syntax without teaching **query thinking**.

Teach them the logical order:

FROM
↓
JOIN
↓
WHERE
↓
GROUP BY
↓
HAVING
↓
SELECT
↓
DISTINCT
↓
ORDER BY
↓
LIMIT

Tell them:

SQL is not written in the same order that SQL logically processes data.

For example:

```
SELECT customer_id,  
       SUM(amount) AS total_spent  
FROM orders  
WHERE order_status = 'Completed'  
GROUP BY customer_id  
HAVING SUM(amount) > 50000  
ORDER BY total_spent DESC;
```

Train them to mentally read it:

```
FROM orders  
↓  
Keep completed orders  
↓  
Group by customer  
↓  
Calculate spending  
↓  
Keep customers > 50K  
↓  
Sort  
↓  
Display
```

This becomes their **SQL execution mental model**.

PHASE 5 — JOIN MASTERY

This is probably the most important transition from beginner → intermediate.

Don't teach JOINS like this:

```
INNER JOIN = matching records  
LEFT JOIN = all records from left table
```

That is technically correct but insufficient.

Teach them using **business questions**.

Scenario

Tables:

CUSTOMERS

customer_id
customer_name

ORDERS

order_id
customer_id
order_amount

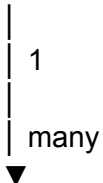
Question:

Show customers and their orders.

First ask:

What relationship exists?

CUSTOMER



ORDERS

Then:

```
SELECT c.customer_name,  
       o.order_amount  
FROM customers c  
JOIN orders o  
  ON c.customer_id = o.customer_id;
```

Then change the business question:

Show customers who have never ordered.

Now they should recognize:

Need customers

+

Need absence of orders

This leads naturally to:

LEFT JOIN

and:

WHERE o.customer_id IS NULL

PHASE 6 — Teach JOINS Through Business Scenarios

Create a progression.

Level 1

Show customers and their orders.

Level 2

Show customers who placed at least one order.

Level 3

Show customers who never placed an order.

Level 4

Show customers with more than 5 orders.

Level 5

Show customers whose total spending exceeds ₹50,000.

Level 6

Show customers who placed an order in every month of 2026.

Level 7

Find customers who purchased product A but never purchased product B.

Now they are no longer practicing syntax.

They're practicing **reasoning**.

PHASE 7 — Scenario-Based Problem Solving

This should become the core of your training.

Instead of:

Write a query using GROUP BY.

Give them:

The sales manager wants to identify customers who are responsible for more than ₹1 lakh in revenue. Only completed orders should be considered.

Now force them to extract:

Business requirement



Completed orders



Customer level



Total revenue



Revenue > 100000

Then SQL:

```
SELECT customer_id,  
       SUM(order_amount) AS revenue  
FROM orders  
WHERE status = 'Completed'  
GROUP BY customer_id  
HAVING SUM(order_amount) > 100000;
```

PHASE 8 — Teach the "Break the Problem Down" Technique

This is probably the **single most important technique** for your trainees.

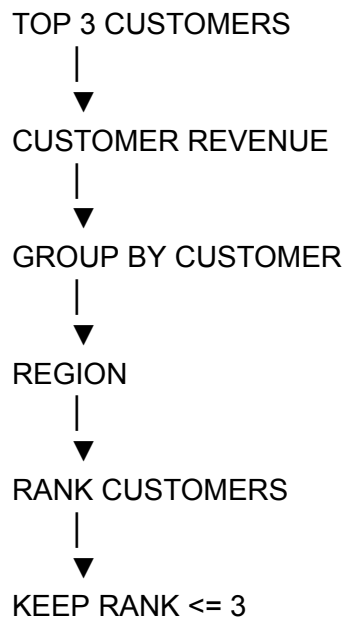
Give them a difficult problem.

Example:

Find the top 3 customers by revenue in each region.

Don't let them write SQL immediately.

Break it down:



Now the SQL concepts reveal themselves:

GROUP BY
+
SUM
+
WINDOW FUNCTION
+
PARTITION BY
+
ORDER BY

This teaches them:

Complex SQL is usually several simple problems connected together.

PHASE 9 — Intermediate SQL

Once they can solve scenarios, introduce:

Subqueries

```
SELECT *  
FROM employees  
WHERE salary >  
    (SELECT AVG(salary)  
     FROM employees);
```

Teach them to ask:

"What information do I need to solve the outer problem?"

CTEs

Then convert:

```
SELECT *  
FROM employees  
WHERE salary >  
    (SELECT AVG(salary)  
     FROM employees);
```

into:

```
WITH avg_salary AS (  
    SELECT AVG(salary) AS avg_sal  
    FROM employees  
)  
SELECT e.*  
FROM employees e  
CROSS JOIN avg_salary a  
WHERE e.salary > a.avg_sal;
```

Explain **why** CTEs exist:

"I'm creating an intermediate result so that my thinking becomes easier to follow."

Not:

"CTE stands for Common Table Expression."

The latter is theory. The former is professional thinking.

PHASE 10 — Window Functions

This should be taught through problems, not syntax.

Start with:

Problem

Rank employees by salary.

```
RANK() OVER (  
  ORDER BY salary DESC  
)
```

Then:

Rank employees within each department.

```
RANK() OVER (  
  PARTITION BY department_id  
  ORDER BY salary DESC  
)
```

Then:

Find the highest-paid employee in each department.

Then:

Find the second-highest salary in each department.

Then:

Find each employee's salary difference from the departmental average.

Then:

Find each customer's previous order.

Now introduce:

LAG()
LEAD()
ROW_NUMBER()
RANK()
DENSE_RANK()
SUM() OVER()
AVG() OVER()

PHASE 11 — Advanced SQL

Now introduce more complex patterns.

1. Recursive thinking

- organizational hierarchies
- employee-manager relationships
- category trees
- bill of materials

2. Recursive CTEs

3. Complex window functions

4. Conditional aggregation

```
SUM(  
  CASE  
    WHEN status = 'Completed'  
    THEN amount  
    ELSE 0  
  END  
)
```

5. Pivoting concepts

6. Set operations

UNION
UNION ALL
INTERSECT
EXCEPT

7. EXISTS / NOT EXISTS

Especially useful for:

Customers who purchased A
but did not purchase B

8. Date problems

This is a major professional skill.

Examples:

- daily sales
 - monthly sales
 - rolling 7-day sales
 - year-over-year growth
 - month-over-month growth
 - customer retention
 - cohort analysis
-

PHASE 12 — Real-World SQL Scenarios

Now stop giving them isolated SQL exercises.

Give them **business problems**.

E-commerce

Find customers who haven't purchased in the last 90 days.

Banking

Identify customers whose transaction volume increased by more than 50% compared with the previous month.

Healthcare

Find patients who missed two consecutive appointments.

HR

Find employees earning above their department average.

Manufacturing

Find machines whose downtime increased for three consecutive months.

Retail

Find the best-selling product in each store.

Finance

Calculate month-over-month revenue growth.

Now trainees have to decide:

What tables?

What columns?

What relationship?

What filters?

What aggregation?

What window?

What intermediate result?

What final result?

That is exactly the thinking you want.

The SQL Problem-Solving Framework

I would make your trainees memorize this.

D → T → R → F → G → W → V

D — Define the requirement

What exactly is being asked?

T — Identify tables

Which tables contain the required information?

R — Identify relationships

How do those tables connect?

F — Filter

Which records should be included?

G — Group

At what level should calculations happen?

W — Window

Do I need ranking, previous/next values, running totals, etc.?

V — Validate

Does the output actually make business sense?

The "Don't Touch SQL Yet" Rule

This is an extremely effective teaching technique.

When you give them a problem:

First 2–5 minutes:

No SQL allowed.

They must write:

Required output:

Tables:

Relationships:

Filters:

Grouping:

Special logic:

Only then can they write SQL.

This prevents the common behavior:

"I'll start writing SELECT and see what happens."

Teach Them to Think in Intermediate Results

Suppose you ask:

Find the top customer in every region.

Don't expect:

SELECT ...

immediately.

Teach:

Step 1

Find customer revenue.

Customer + Region + Revenue

Step 2

Rank customers inside each region.

Customer + Region + Revenue + Rank

Step 3

Keep rank = 1.

Customer + Region + Revenue

This mindset is extremely powerful.

Tell them:

If you cannot explain your intermediate table, you probably don't understand your SQL yet.

The "Result Shape" Technique

This is another powerful technique.

Before writing SQL, ask:

What should one row represent?

Examples:

Customer report

One row = one customer

Monthly sales report

One row = one month

Product performance

One row = one product

Customer + product report

One row = one customer-product combination

This prevents many **GROUP BY** and JOIN errors.

Teach Cardinality

Professional SQL developers think about:

1 → 1

1 → many

many → 1

many → many

Before every JOIN, ask:

How many rows can this JOIN create?

Example:

Customer

1

|

many



Orders

If they don't understand this, they will produce duplicate revenue.

This is one of the biggest differences between beginner and professional SQL.

Teach "Duplicate Detection"

Give them deliberately broken SQL.

Example:

```
SELECT SUM(o.amount)
```

```
FROM orders o
JOIN order_items oi
  ON o.order_id = oi.order_id;
```

Ask:

Is this revenue correct?

This creates a very important realization:

1 order
↓
5 order items
↓
order appears 5 times

Therefore:

JOINS change the grain of your data.

That concept should be emphasized heavily.

The SQL Debugging Method

Teach them that SQL development is not:

Write query → Done

Instead:

Write
↓
Run
↓
Inspect
↓
Question
↓
Debug
↓
Modify
↓
Validate

When a query returns 10,000 rows instead of 1,000:

Don't fix it randomly.

Ask:

1. Did the JOIN multiply rows?
2. Is the WHERE condition correct?
3. Is the GROUP BY at the correct grain?
4. Did I accidentally use INNER JOIN?
5. Did NULL values affect the result?
6. Did aggregation happen before/after the JOIN?

This creates professional habits.

A Very Effective Classroom Structure

For every class, use:

10% — Concept

Explain one concept.

20% — Demonstration

Solve one problem yourself.

But **think aloud**.

Say:

"The question says each department, so I know my output grain is department."

"I need salary, which is in employees."

"I need department name, which is in departments."

"Therefore I need a JOIN."

This is called **worked-example thinking** and is extremely useful for beginners.

30% — Guided Practice

Give a similar problem.

Help them with questions rather than answers.

Instead of:

"Use LEFT JOIN."

Ask:

"What should happen to customers who have no orders?"

They eventually say:

"They should still appear."

Then:

"Which JOIN preserves them?"

Now they discovered LEFT JOIN.

30% — Independent Scenario

Give a new problem.

No hints.

10% — Review

Discuss:

- different approaches
 - mistakes
 - edge cases
 - performance
 - readability
-

Use the "Hint Ladder"

When someone gets stuck, **don't immediately give the answer**.

Use levels.

Hint 1 — Requirement

What exactly are we trying to return?

Hint 2 — Tables

Which table contains that information?

Hint 3 — Relationship

How can those tables be connected?

Hint 4 — Grain

What should one row represent?

Hint 5 — Operation

Are you filtering, grouping, or ranking?

Hint 6 — SQL concept

Would a window function help here?

Hint 7 — Syntax

Only now show syntax.

This teaches **independence**.

Create Difficulty Levels

Don't randomly mix questions.

Use a structured ladder.

Level 1 — Recall

SELECT
WHERE
ORDER BY

Example:

Find customers from Mumbai.

Level 2 — Combine

WHERE + ORDER BY

Example:

Find Mumbai customers who spent more than ₹10,000.

Level 3 — Aggregate

GROUP BY + aggregate

Example:

Find total spending by customer.

Level 4 — Multi-table

JOIN + GROUP BY

Example:

Find total spending by customer name.

Level 5 — Conditional logic

CASE

Level 6 — Subquery

Customers spending above average.

Level 7 — CTE

Break complex logic into stages.

Level 8 — Window function

Ranking, running totals, LAG/LEAD.

Level 9 — Multi-step scenario

Several concepts together.

Level 10 — Real-world case study

No indication of which SQL concept to use.

This is where they become problem solvers.

Your Training Dataset Matters

Don't keep changing datasets every day.

Use **one realistic database** for several weeks.

For example:

CUSTOMERS
PRODUCTS
ORDERS
ORDER_ITEMS
PAYMENTS
EMPLOYEES
STORES

Then build increasingly difficult questions around it.

This is important because trainees should spend their cognitive energy on:

solving the problem

rather than:

learning another database schema.

Introduce "Business Language → SQL Language"

This is one of the most valuable professional skills.

Teach them to translate:

Business phrase	SQL thinking
each customer	GROUP BY customer
total	SUM
average	AVG

highest	MAX / RANK
top 3	ROW_NUMBER / RANK
never	LEFT JOIN / NOT EXISTS
at least	>=
every	relational logic / counts
previous	LAG
next	LEAD
percentage	numerator / denominator
compared with previous month	LAG
growing continuously	window/date logic
above average	subquery / CTE
latest	MAX / ROW_NUMBER
first purchase	ROW_NUMBER / MIN

This becomes a **SQL vocabulary dictionary** in their heads.

Teach Multiple Solutions

Once they solve a problem, ask:

Can you solve this another way?

For example:

Customers without orders

Method 1:

LEFT JOIN

Method 2:

NOT EXISTS

Method 3:

NOT IN

Then discuss:

- readability
- NULL behavior
- performance
- maintainability

This develops maturity.

Add "What's Wrong With This Query?" Exercises

These are incredibly effective.

Instead of only asking them to write SQL, give them broken SQL.

Example:

```
SELECT customer_id,  
       customer_name,  
       SUM(amount)  
FROM customers c  
JOIN orders o  
  ON c.customer_id = o.customer_id  
WHERE SUM(amount) > 50000  
GROUP BY customer_id;
```

Ask:

Find as many problems as possible.

They might identify:

- `SUM()` cannot be used in `WHERE`
- `customer_name` needs grouping
- output alias could improve readability

This develops debugging ability.

Add "Predict the Output" Exercises

Give them a small dataset.

Then ask:

What will this query return?

Before executing it.

This develops mental execution.

For example:

```
SELECT customer_id,  
       COUNT(*) AS orders  
FROM orders  
GROUP BY customer_id  
HAVING COUNT(*) > 2;
```

Ask them to predict the result.

Then run it.

This is extremely useful for developing SQL intuition.

Teach Edge Cases

Professional SQL developers don't only ask:

"Does my query work?"

They ask:

"When could this query fail or produce misleading results?"

Introduce:

NULL
duplicates
missing records
zero values
negative values
multiple orders
same timestamps
ties
empty tables
duplicate keys

For example:

What happens if two employees have exactly the same salary?

Now they need to understand:

ROW_NUMBER
RANK
DENSE_RANK

Advanced Professional Thinking

Eventually train them to ask these questions automatically:

Data

What is the grain?

Relationship

Can this JOIN multiply rows?

Logic

What exactly is being measured?

NULL

How will NULL affect this calculation?

Performance

How much data is being processed?

Maintainability

Will another developer understand this query?

Validation

How do I know the result is correct?

This is **professional SQL thinking**.

SQL Code Review Training

Towards the end, introduce code reviews.

Give them a query and ask them to review it for:

- Correctness
- Readability
- Performance
- Maintainability
- Edge cases
- Naming
- Duplicates
- NULL handling

Don't simply ask:

"Is the query correct?"

Ask:

"Would you approve this query in production?"

That's a completely different mindset.

Suggested 10-Week Curriculum

Week	Focus	Main Skill
1	SELECT, WHERE, expressions	Query basics
2	Aggregation, GROUP BY, HAVING	Thinking in groups
3	JOINS	Relationships
4	JOIN + aggregation	Multi-table reasoning
5	CASE, subqueries, EXISTS	Conditional logic
6	CTEs	Breaking problems down
7	Window functions	Analytical thinking
8	Dates + advanced scenarios	Business problems
9	Complex case studies	Problem solving
10	Optimization + code review	Professional SQL

The Daily Practice Model

I strongly recommend this structure.

Problem 1 — Warm-up

Easy:

Find all active customers.

Problem 2 — Combine

Find active customers with orders above ₹10,000.

Problem 3 — Scenario

Identify customers whose spending exceeds the average customer spending.

Problem 4 — Challenge

Find the top 3 customers by revenue in each region.

Problem 5 — Debug

Give them a broken query.

Problem 6 — Explain

Ask:

Explain your query without reading it line by line.

This last exercise is underrated.

If they cannot explain their query in plain English, they probably don't fully understand it.

The "SQL Journal"

Have each trainee maintain a small problem-solving journal.

For every difficult question, record:

Problem:

What was the requirement?

What should one row represent?

Tables:

Relationships:

Filters:

Aggregation:

SQL concepts used:

My first mistake:

Final solution:

Alternative solution:

What I learned:

After 50–100 problems, this becomes an extremely valuable learning resource.

The Most Important Teaching Rule

Don't rescue them too quickly.

If a trainee says:

"Sir/Ma'am, I don't know how to do this."

Don't respond:

"Use a CTE."

Ask:

"Tell me what the question is asking."

Then:

"What should one row represent?"

Then:

"Which tables do you need?"

Then:

"How are they related?"

Then:

"What calculation do you need?"

By the time you ask these questions, **they often discover the solution themselves.**

That's the real goal.

What You Should Stop Doing

Avoid these common training mistakes:

- ✗ Teaching 20 SQL functions in one session.
- ✗ Giving 30 questions that all look identical.
- ✗ Telling students which SQL concept to use.

"Write a query using a window function..."

Instead:

"Find the second-highest employee in each department."

Let them discover the window function.

- ✗ Giving the answer immediately.
 - ✗ Focusing only on syntax.
 - ✗ Rewarding only correct answers.
 - ✗ Ignoring incorrect approaches.
 - ✗ Using only toy datasets.
 - ✗ Teaching advanced SQL before basic problem decomposition.
-

What You Should Start Doing

Instead:

1. Teach thinking before syntax.
 2. Give business scenarios.
 3. Force them to identify the grain.
 4. Make them draw table relationships.
 5. Make them predict results.
 6. Make them explain their query.
 7. Give broken queries.
 8. Ask for alternative solutions.
 9. Discuss edge cases.
 10. Introduce performance thinking.
 11. Use progressive difficulty.
 12. Make them solve unfamiliar problems.
-

The Final Goal

Your trainees should eventually see a question like:

"Find customers whose monthly spending increased for three consecutive months, but only consider customers who made at least five purchases during the period."

And instead of thinking:

"Oh no, this is advanced SQL."

They should automatically think:

What is the required output?



One row per customer?



What data do I need?



Customer + orders



What is the grain?



Customer-month



Calculate monthly spending



Use LAG()



Compare current vs previous month



Identify consecutive increases



Count purchases



Filter customers



Validate results

That transformation—from "What SQL syntax do I know?" to "How do I decompose this problem?"—is what will turn your trainees into SQL developers.

My recommended teaching philosophy

Use this formula throughout the entire program:

Business Problem → Understand → Decompose → Identify Grain → Identify Tables → Identify Relationships → Build Intermediate Results → Write SQL → Test → Validate → Optimize → Explain

If you consistently train them this way, **SQL becomes the language they use to express their reasoning**, rather than something they memorize.

And I would make one rule almost sacred in your classroom:

"Don't start with SELECT. Start with the problem."